

Universidad Nacional de Colombia

Programación Orientada a Objetos 2026-2S

Actividad 3: Valor 10%

Estudiante

Juan Diego Toro Zuluaga
C.C. 1022143298

Docente

Walter Hugo Arboleda Mazo

Repositorio GitHub

https://github.com/jtoroz/Actividades_Poo

22 de octubre de 2026

Ejercicio 8.2 - Notas

Enunciado. Se requiere un programa con interfaz gráfica de usuario que solicite las cinco notas obtenidas por un estudiante. Al pulsar el botón Calcular, el programa calcula y muestra en la parte inferior de la ventana el promedio de las notas, su desviación estándar, la mayor nota y la menor nota. Además debe tener un botón Limpiar que vacíe todos los campos de texto.

URL en GitHub: https://github.com/jtoroz/Actividades_Poo/blob/main/actividad3_ej82_notas.py

Código fuente (actividad3_ej82_notas.py)

```
# Actividad 3 - Ejercicio 8.2: Notas
# Programacion Orientada a Objetos 2026-2S - Juan Diego Toro Zuluaga

import math
import tkinter as tk

class Notas:

    def __init__(self, lista_notas):
        self.lista_notas = lista_notas

    def calcular_promedio(self):
        return sum(self.lista_notas) / len(self.lista_notas)

    def calcular_desviacion(self):
        promedio = self.calcular_promedio()
        suma = 0
        for nota in self.lista_notas:
            suma += (nota - promedio) ** 2
        return math.sqrt(suma / len(self.lista_notas))

    def calcular_mayor(self):
        mayor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota > mayor:
                mayor = nota
        return mayor

    def calcular_menor(self):
        menor = self.lista_notas[0]
        for nota in self.lista_notas:
            if nota < menor:
                menor = nota
        return menor

class VentanaPrincipal(tk.Tk):

    def __init__(self):
        super().__init__()
        self.title("Notas")
        self.resizable(False, False)

        self.campos = []
        for i in range(5):
            tk.Label(self, text="Nota " + str(i + 1)).grid(row=i, column=0, padx=10,
                pady=5, sticky="w")
            campo = tk.Entry(self, width=12)
```

```

        campo.grid(row=i, column=1, padx=10, pady=5)
        self.campos.append(campo)

tk.Button(self, text="Calcular", width=10,
          command=self.calcular).grid(row=5, column=0, padx=10, pady=10)
tk.Button(self, text="Limpiar", width=10,
          command=self.limpiar).grid(row=5, column=1, padx=10, pady=10)

self.etiqueta_promedio = tk.Label(self, text="Promedio =")
self.etiqueta_promedio.grid(row=6, column=0, columnspan=2, padx=10, sticky="w")

self.etiqueta_desviacion = tk.Label(self, text="Desviación estándar =")
self.etiqueta_desviacion.grid(row=7, column=0, columnspan=2, padx=10, sticky="w")

self.etiqueta_mayor = tk.Label(self, text="Mayor nota =")
self.etiqueta_mayor.grid(row=8, column=0, columnspan=2, padx=10, sticky="w")

self.etiqueta_menor = tk.Label(self, text="Menor nota =")
self.etiqueta_menor.grid(row=9, column=0, columnspan=2, padx=10, pady=(0, 10),
                          sticky="w")

def calcular(self):
    lista_notas = []
    for campo in self.campos:
        lista_notas.append(float(campo.get()))

    notas = Notas(lista_notas)

    self.etiqueta_promedio.config(text="Promedio = %.2f" % notas.calcular_promedio())
    self.etiqueta_desviacion.config(text="Desviación estándar = %.2f" %
                                     notas.calcular_desviacion())
    self.etiqueta_mayor.config(text="Mayor nota = %.2f" % notas.calcular_mayor())
    self.etiqueta_menor.config(text="Menor nota = %.2f" % notas.calcular_menor())

def limpiar(self):
    for campo in self.campos:
        campo.delete(0, tk.END)
    self.etiqueta_promedio.config(text="Promedio =")
    self.etiqueta_desviacion.config(text="Desviación estándar =")
    self.etiqueta_mayor.config(text="Mayor nota =")
    self.etiqueta_menor.config(text="Menor nota =")

ventana = VentanaPrincipal()
ventana.mainloop()

```

Interfaz de usuario

Nota 1	4.5
Nota 2	3.8
Nota 3	2.9
Nota 4	4.0
Nota 5	3.2

Calcular

Limpiar

Promedio = 3.68
 Desviación estándar = 0.57
 Mayor nota = 4.50
 Menor nota = 2.90

Ventana del programa Notas con las cinco notas calculadas

Diagrama de clases

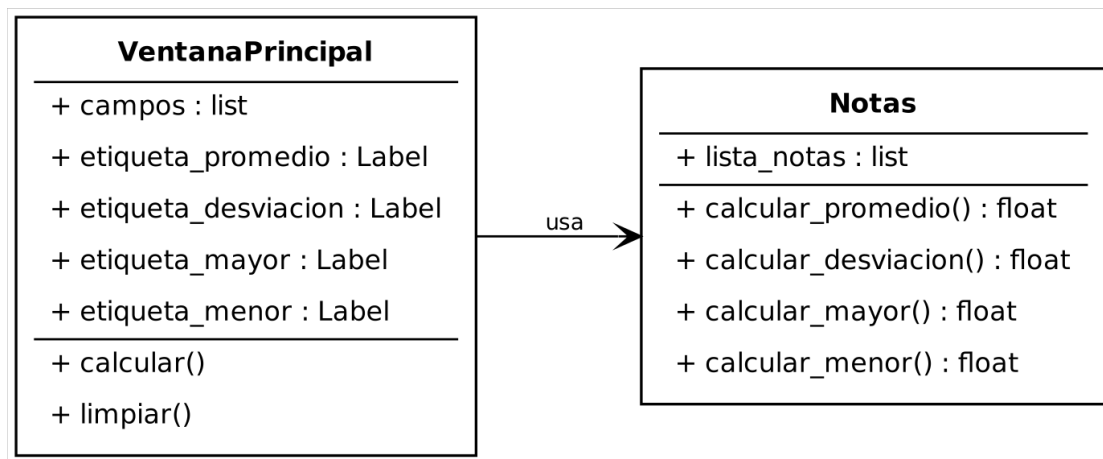


Diagrama de objetos

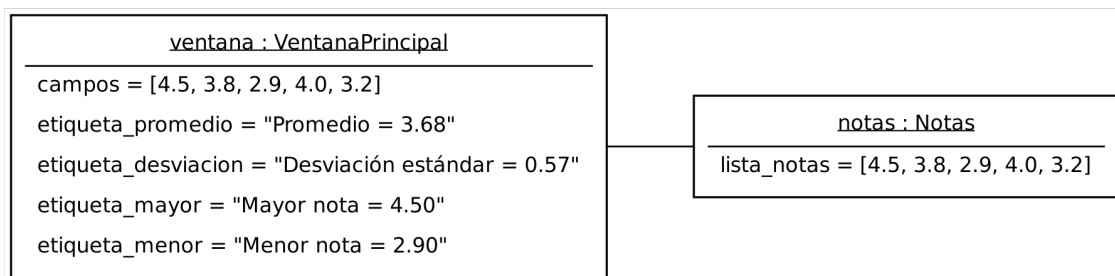
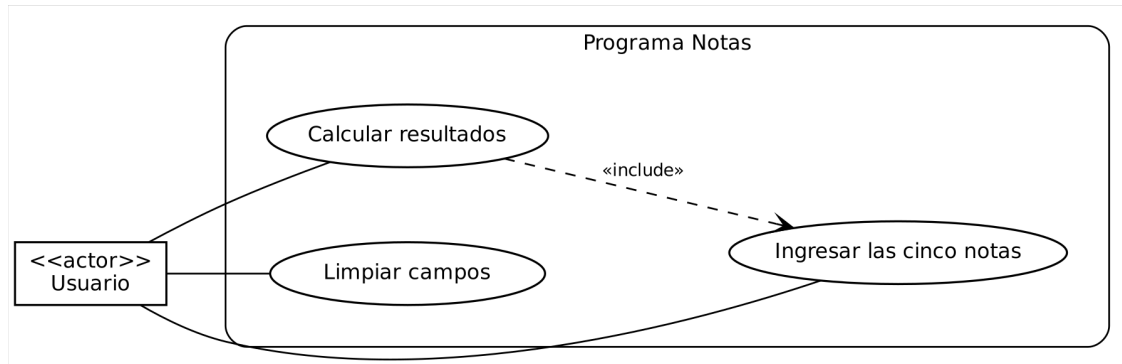


Diagrama de casos de uso



Ejercicio 8.3 - Figuras

Enunciado. Se requiere un programa con interfaz gráfica de usuario que calcule el volumen y la superficie de tres figuras geométricas tridimensionales: cilindro (radio y altura), esfera (radio) y pirámide (base, altura y apotema), todas en centímetros. Las figuras se modelan con una jerarquía de clases que hereda de una clase padre FiguraGeometrica. La ventana principal presenta un botón por figura y cada uno abre la ventana correspondiente. Si se deja un campo vacío o se escribe un valor que no es numérico, se captura la excepción y se muestra una ventana emergente de error.

URL en GitHub: https://github.com/jtoroz/Actividades_Poo/blob/main/actividad3_ej83_figuras.py

Código fuente (actividad3_ej83_figuras.py)

```
# Actividad 3 - Ejercicio 8.3: Figuras
# Programacion Orientada a Objetos 2026-2S - Juan Diego Toro Zuluaga

import math
import tkinter as tk
from tkinter import messagebox

class FiguraGeometrica:

    def __init__(self):
        self.volumen = 0
        self.superficie = 0

    def get_volumen(self):
        return self.volumen

    def set_volumen(self, volumen):
        self.volumen = volumen

    def get_superficie(self):
        return self.superficie

    def set_superficie(self, superficie):
        self.superficie = superficie

class Cilindro(FiguraGeometrica):

    def __init__(self, radio, altura):
        super().__init__()
        self.radio = radio
        self.altura = altura

    def calcular_volumen(self):
        self.volumen = math.pi * self.radio ** 2 * self.altura
        return self.volumen

    def calcular_superficie(self):
        self.superficie = 2 * math.pi * self.radio * self.altura + 2 * math.pi * self.radio
        ** 2
        return self.superficie

class Esfera(FiguraGeometrica):
```

```

def __init__(self, radio):
    super().__init__()
    self.radio = radio

def calcular_volumen(self):
    self.volumen = (4 / 3) * math.pi * self.radio ** 3
    return self.volumen

def calcular_superficie(self):
    self.superficie = 4 * math.pi * self.radio ** 2
    return self.superficie

class Piramide(FiguraGeometrica):

    def __init__(self, base, altura, apotema):
        super().__init__()
        self.base = base
        self.altura = altura
        self.apotema = apotema

    def calcular_volumen(self):
        self.volumen = self.base ** 2 * self.altura / 3
        return self.volumen

    def calcular_superficie(self):
        self.superficie = self.base ** 2 + 2 * self.base * self.apotema
        return self.superficie

class VentanaFigura(tk.Toplevel):
    """Ventana base: arma los campos de entrada y las etiquetas de resultado."""

    def __init__(self, titulo, nombres_campos):
        super().__init__()
        self.title(titulo)
        self.resizable(False, False)

        self.campos = []
        for i, nombre in enumerate(nombres_campos):
            tk.Label(self, text=nombre + " (cm)").grid(row=i, column=0, padx=10, pady=5,
                sticky="w")
            campo = tk.Entry(self, width=12)
            campo.grid(row=i, column=1, padx=10, pady=5)
            self.campos.append(campo)

        fila = len(nombres_campos)
        tk.Button(self, text="Calcular", width=10,
            command=self.calcular).grid(row=fila, column=0, columnspan=2, pady=10)

        self.etiqueta_volumen = tk.Label(self, text="Volumen =")
        self.etiqueta_volumen.grid(row=fila + 1, column=0, columnspan=2, padx=10,
            sticky="w")

        self.etiqueta_superficie = tk.Label(self, text="Superficie =")
        self.etiqueta_superficie.grid(row=fila + 2, column=0, columnspan=2, padx=10,
            pady=(0, 10), sticky="w")

    def crear_figura(self, valores):
        pass

    def calcular(self):
        try:

```

```

        valores = [float(campo.get()) for campo in self.campos]
    except ValueError:
        messagebox.showerror("Error", "Campo nulo o error en formato de número")
        return

    figura = self.crear_figura(valores)
    self.etiqueta_volumen.config(text="Volumen = %.2f cm3" % figura.calcular_volumen())
    self.etiqueta_superficie.config(text="Superficie = %.2f cm2" %
        figura.calcular_superficie())

class VentanaCilindro(VentanaFigura):

    def __init__(self):
        super().__init__("Cilindro", ["Radio", "Altura"])

    def crear_figura(self, valores):
        return Cilindro(valores[0], valores[1])

class VentanaEsfera(VentanaFigura):

    def __init__(self):
        super().__init__("Esfera", ["Radio"])

    def crear_figura(self, valores):
        return Esfera(valores[0])

class VentanaPiramide(VentanaFigura):

    def __init__(self):
        super().__init__("Pirámide", ["Base", "Altura", "Apotema"])

    def crear_figura(self, valores):
        return Piramide(valores[0], valores[1], valores[2])

class VentanaPrincipal(tk.Tk):

    def __init__(self):
        super().__init__()
        self.title("Figuras")
        self.resizable(False, False)

        tk.Label(self, text="Seleccione una figura").pack(padx=20, pady=10)
        tk.Button(self, text="Cilindro", width=12, command=VentanaCilindro).pack(padx=20,
            pady=5)
        tk.Button(self, text="Esfera", width=12, command=VentanaEsfera).pack(padx=20,
            pady=5)
        tk.Button(self, text="Pirámide", width=12, command=VentanaPiramide).pack(padx=20,
            pady=(5, 15))

ventana = VentanaPrincipal()
ventana.mainloop()

```

Interfaz de usuario

Seleccione una figura

Cilindro

Esfera

Pirámide

Ventana principal con el menú de figuras

Radio (cm)

5

Altura (cm)

12

Calcular

Volumen = 942.48 cm³

Superficie = 534.07 cm²

Ventana del cilindro con el volumen y la superficie calculados

Base (cm)

6

Altura (cm)

9

Apotema (cm)

7.5

Calcular

Volumen = 108.00 cm³

Superficie = 126.00 cm²

Ventana de la pirámide con el volumen y la superficie calculados

Diagrama de clases

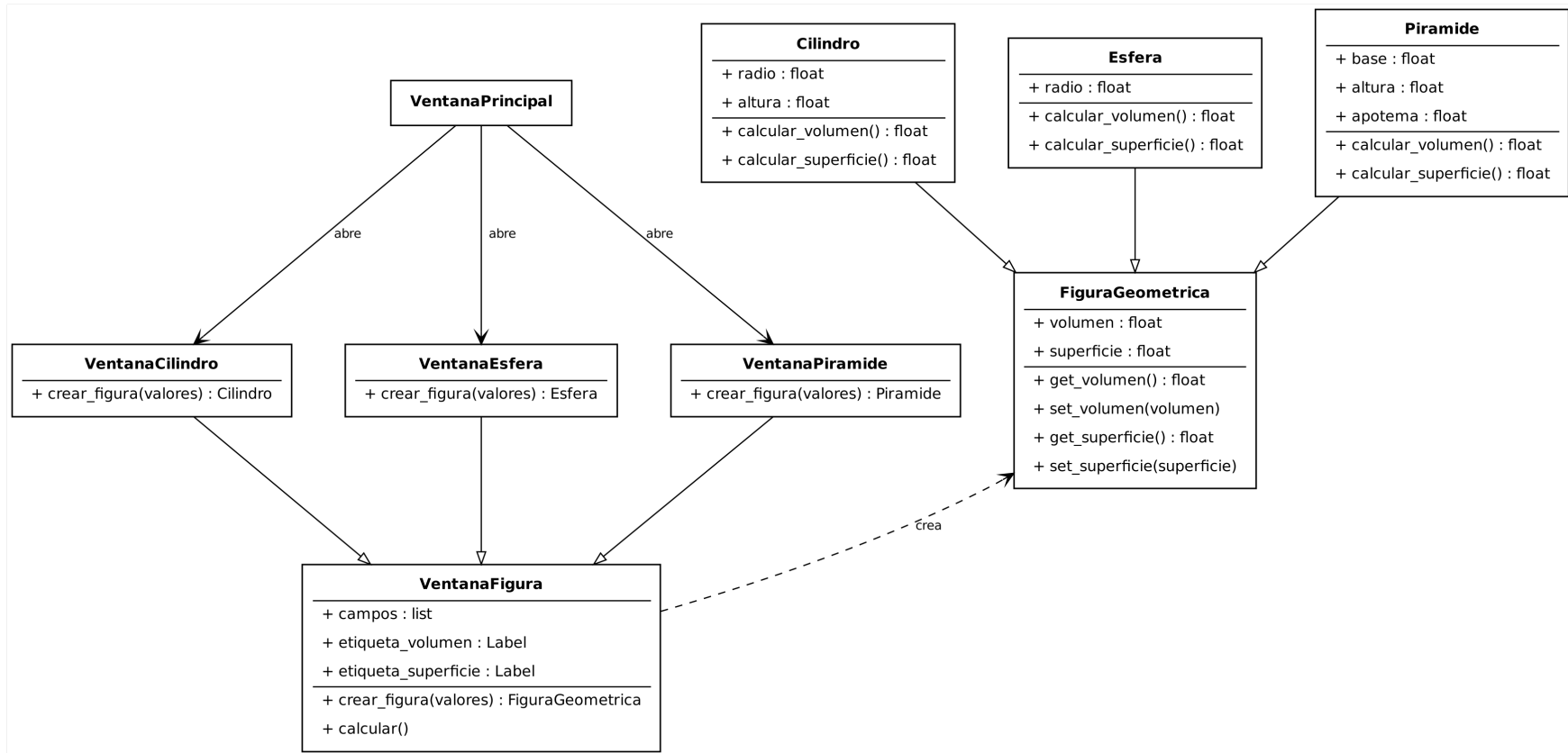


Diagrama de objetos

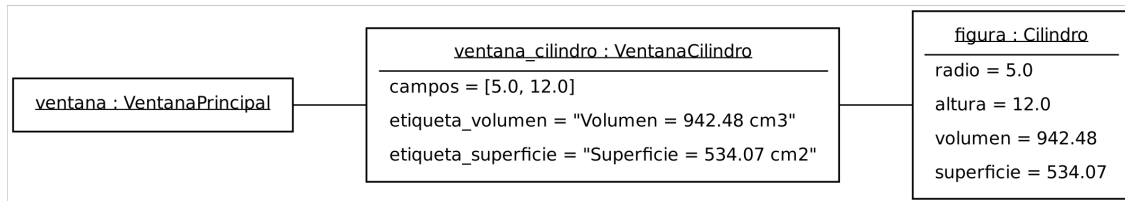


Diagrama de casos de uso

